Danilets Website

Technical Optimization Report

Project: danilets.com / daniletsdetailing.com / daniletscleaning.com

Scope: Performance · Code Quality · SEO · Backend Architecture

Status: Completed

Area	Before	After
Total image payload	~700 MB	~22 MB (–97%)
Largest hero slide	11 MB JPEG	459 KB WebP
JS on first page load	All routes	Home page only
Booking form JS	Loaded upfront	On-demand
server.js size	2,343 lines	170 lines
Pages with unique SEO	0	11
JSON-LD structured data	None	3 schemas
Phone / email in code	Hardcoded	CRM-editable

01.

Media Optimization (Images & Assets)

Problem

The website carried over 700 MB of uncompressed images. Hero slides alone included an 11 MB JPEG and a 5.8 MB JPEG. Every visitor was forced to download massive files before seeing the page.

What we did

- Converted all JPG/PNG images to **WebP format** across four directories — Portfolio (372 MB → 3.9 MB), Portfolio Cleaning (129 MB → 13.8 MB), Services (97 MB → 2.8 MB), and site assets (45 MB → 2.1 MB).
- Updated all React component imports and inline image paths to reference the new .webp files.
- Added **loading="lazy"** to all below-the-fold images (portfolio, team, services).
- Added **fetchPriority="high"** and **loading="eager"** to the first hero slide (the Largest Contentful Paint element) so the browser prioritizes it immediately.
- Cleared all original JPG/PNG files that are no longer referenced, reducing repository weight by ~46 MB in source assets alone.

Directory	Before	After	Reduction
Portfolio/	372 MB	3.9 MB	99%
Portfolio_Cleaning/	129 MB	13.8 MB	89%
Services/	97 MB	2.8 MB	97%
src/assets/photo/	45 MB	2.1 MB	95%
Hero slides total	~23 MB	~1.2 MB	95%

02.

JavaScript Code Splitting & Lazy Loading

Problem

Every route — including the entire multi-step booking form with 36 imported step components — was bundled and downloaded on the very first page load, even if the visitor never opened the booking page.

What we did

- Converted all 11 page-level routes in App.jsx to **React.lazy()** with Suspense. Only the home page components remain eagerly loaded.
- Converted all **36 booking form step components** in Booking.jsx to React.lazy(). Steps load only when the user navigates to them during the booking flow.
- Configured **manualChunks** in vite.config.js to split the bundle into 8 separate vendor chunks: vendor-react, vendor-router, vendor-swiper, vendor-stripe, vendor-icons, vendor-google-auth, vendor-whop, vendor-axios.

Each chunk is independently cached by the browser.

Result: On first visit, only React core + home page code downloads. The entire booking flow (~400 KB JS) loads only when the user navigates to /book-online. Stripe and Swiper never load unless needed.

03.

Dead Code & Duplicate Asset Cleanup

Problem

The codebase contained obsolete files from earlier development iterations alongside duplicated assets scattered across multiple directories.

What we did

- Identified 7 outdated booking step components superseded by current versions. Confirmed none were still imported, then cleared their content (~54 KB of dead code).
- Found the company logo SVG duplicated in 4 separate locations. Cleared the 3 unused copies; all components import from the single canonical source.
- Removed 6 PNG icon files that had SVG equivalents and were not imported anywhere.
- Cleared all 26 original JPG/PNG files from src/assets/photo/ — no longer referenced by any component after WebP conversion.

04.

Backend Refactoring

Problem

The Express server was a single 2,343-line monolithic file containing authentication, database models, Bitrix24 CRM integration, request handling, push notifications, and static file serving — all mixed together. This made the codebase difficult to maintain, debug, or extend.

What we did

Decomposed server.js into 9 focused modules, each with comments explaining its purpose:

File	Lines	Responsibility
server.js	170	App setup, middleware, route mounting, startup
middleware/authMiddleware.js	68	JWT auth, optionalAuth, requireAdmin, signToken
helpers/bitrix.js	398	All Bitrix24 CRM integration logic
routes/authRoutes.js	280	OTP, register, login, password reset, Apple Sign In
routes/contentRoutes.js	98	Content blocks CRUD
routes/profileRoutes.js	125	User profile, vehicles, payment methods
routes/requestRoutes.js	175	Service requests (user & guest)
routes/pushRoutes.js	53	Web push notification subscriptions
routes/adminRequestRoutes.js	119	Admin: manage all requests

All 39 API endpoints preserved with identical behavior. Zero functionality changed.

05.

SEO Optimization

Problem

react-helmet-async was installed as a dependency but never connected. Every page — detailing, cleaning, about us, booking — showed the same generic title "Danilets Family" in Google search results. No Open Graph tags, no structured data, no canonical URLs.

What we did

- Connected **HelmetProvider** in main.jsx (required for react-helmet-async to function).
- Updated index.html with base Open Graph tags, Twitter Card fallback, geo meta tags for local SEO (geo.region: US-OH, geo.placename: Columbus, Ohio), and preconnect for Google Fonts.
- Created a reusable **SEO.jsx** component that automatically detects which domain is active and adjusts the siteName accordingly. Applied unique titles and descriptions to all 11 pages.

- Created three **Schema.org LocalBusiness JSON-LD schemas** for the main, detailing, and cleaning pages — including business name, phone, email, address, geo coordinates, opening hours, area served, and a complete service catalog.

Page	Title	noIndex
Home (main)	Premium Auto Detailing & Commercial Cleaning	—
Detailing	Premium Auto Detailing — Columbus, OH	—
Cleaning	Commercial & Residential Cleaning — Columbus, OH	—
About Us	About Us — Danilets Family	—
Book Online	Book Online — Auto Detailing & Cleaning	—
Contact	Contact Us	—
Legal	Legal — Privacy Policy & Terms	Yes
Booking Success	Booking Confirmed	Yes
My Account	My Account	Yes
Admin	Admin — CRM Panel	Yes

06.

Dynamic Business Settings

Problem

Business contact details (phone number, email, address) were hardcoded in the structured data. Updating them required a code change and full redeployment.

What we did

- Stored phone, email, and address as **ContentBlock** records in MongoDB (keys: business_phone, business_email, business_address).
- Created **useBusinessInfo()** — a React hook that fetches these values from the API on page load, with a 5-minute in-memory cache and graceful fallback to default values.
- Updated all three StructuredData schemas to use the dynamic hook values.
- Fixed a bug in contentApi in api.js where list() and save() were incorrectly pointing to /api/admin/requests instead of /api/content.
- Created **AdminSettings page (/admin/settings)** — accessible from the CRM panel sidebar (Business Settings button). Three editable fields with inline save feedback and automatic cache invalidation. Protected: redirects non-admins automatically.

Flow: Admin edits phone → clicks Save → value updates in MongoDB → useBusinessInfo cache clears → next page render picks up the new value → Google re-crawls and updates structured data (24–48 hours).

Summary

Area	Before	After
Total image payload (public/)	~700 MB	~22 MB (–97%)
Largest hero slide weight	11 MB JPEG	459 KB WebP
JS on first page load	All routes	Home page only
Booking form JS	Loaded upfront	On-demand per step
server.js size	2,343 lines	170 lines
Pages with unique SEO	0	11
Structured data (JSON-LD)	None	3 schemas
Phone / email in code	Hardcoded	CRM-editable
Dead code / duplicate assets	Present	Cleared

All changes were made without breaking existing functionality. Every API endpoint, booking flow, authentication method, and CRM integration remains intact and operational.